Mô tả đề tài

ĐỀ TÀI: THÙNG RÁC AI TỰ PHÂN LOẠI SỬ DỤNG JETSON NANO VÀ DEEP LEARNING

> **Tên tiếng Anh đề xuất:** Deep Learning-Based Automatic Waste Sorting Bin on NVIDIA Jetson Nano
>
> **Hướng:** Edge AI · Computer Vision · Deep Learning · Embedded System · Cơ điện tử
>
> **Quy mô:** Nhóm 4 sinh viên

---

1. Mô tả đề tài

Đề tài xây dựng một **thùng rác thông minh có khả năng tự nhận diện và phân loại rác bằng camera và mô hình Deep Learning**, sau đó điều khiển cơ cấu cơ khí để đưa rác vào đúng ngăn.

Khác với thùng rác thông thường yêu cầu người dùng tự xác định loại rác, hệ thống đề xuất chỉ cần người dùng đưa một vật rác vào **vùng nhận diện**. Camera chụp ảnh, Jetson Nano chạy mô hình nhận diện, xác định loại rác và gửi lệnh cho vi điều khiển để điều khiển servo/cơ cấu phân luồng.

1.1. Phạm vi phiên bản đầu tiên

Để đề tài khả thi trên Jetson Nano, phiên bản đầu nên giới hạn ở **4 nhóm rác**:

- **Plastic** – chai/hộp nhựa.
- **Paper/Cardboard** – giấy, bìa carton.
- **Metal** – lon kim loại.
- **Other** – các vật không thuộc ba nhóm trên.

> **Không nên bắt đầu với quá nhiều lớp.** TACO có taxonomy rất chi tiết nhưng dữ liệu giữa các lớp không cân bằng. Paper TACO cũng lưu ý dataset còn nhỏ đối với bài toán triển khai thực tế. Vì vậy nhóm nên gộp các nhãn chi tiết thành một số nhóm lớn phù hợp với cơ cấu thùng.

1.2. Mục tiêu chính

1. Xây dựng bộ dữ liệu rác phù hợp với điều kiện camera/thùng rác thực tế.
2. Huấn luyện mô hình Deep Learning nhận diện loại rác.
3. Tối ưu mô hình để inference trực tiếp trên Jetson Nano.
4. Thiết kế cơ cấu tự động phân rác vào các ngăn.
5. Tích hợp AI, camera, cảm biến, Jetson Nano và vi điều khiển thành prototype hoàn chỉnh.
6. Đánh giá đồng thời **độ chính xác AI** và **hiệu năng hệ thống thực tế**.

---

2. Các công trình liên quan

Paper 1 – WasteNet: Waste Classification at the Edge for Smart Bins

- **White et al., 2020.**
- Bài toán: phân loại rác trực tiếp trên thiết bị edge.
- Mô hình dựa trên CNN, phân loại 6 nhóm: paper, cardboard, glass, metal, plastic và other.
- Tác giả định hướng triển khai trên thiết bị công suất thấp như **Jetson Nano**.
- Báo cáo độ chính xác test khoảng **97%**.
- **Giá trị với đề tài:** chứng minh bài toán phân loại rác tại edge là khả thi và là baseline tốt cho hướng classification.

Paper 2 – TACO: Trash Annotations in Context for Litter Detection

- **Proença & Simões, 2020.**
- Giới thiệu dataset **TACO – Trash Annotations in Context**.
- 1.500 ảnh và 4.784 annotation trong phiên bản được mô tả trong paper.
- Rác xuất hiện trong môi trường thực tế thay vì chỉ trên nền trắng.

- Annotation theo COCO, hỗ trợ object detection/instance segmentation.
- Paper thử nghiệm Mask R-CNN.
- **Giá trị với đề tài:** nguồn dữ liệu phù hợp nếu nhóm chọn **object detection** thay vì chỉ image classification.

Paper 3 – Evaluation of Practical Edge Computing CNN-based Solutions for Intelligent Recycling Bins

- **Li & Grammenos, 2023.**
- Xây dựng smart recycling bin và đánh giá các giải pháp CNN trên thiết bị edge, trong đó có Jetson Nano.
- Nhóm tác giả thu thêm khoảng 1.872 ảnh trong môi trường thùng rác thực tế vì dữ liệu TrashNet nền trắng không đại diện tốt cho điều kiện triển khai.
- Đánh giá cả accuracy và power consumption.
- **Bài học quan trọng:** nhóm mình cũng nên tự chụp thêm dữ liệu bằng chính camera và buồng nhận diện của prototype.

Paper 4 – Classification and Recycling of Recyclable Garbage Based on Deep Learning

- **Chen et al., 2023, Journal of Cleaner Production.**
- Sử dụng YOLOv5s và cải tiến theo hướng lightweight bằng ShuffleNet v2/depthwise separable convolution.
- Dataset gồm 10 loại rác tái chế.
- Mô hình cải tiến được triển khai trên **NVIDIA Jetson Nano**.
- Bài báo báo cáo mAP khoảng 94%, số tham số giảm và inference nhanh hơn YOLOv5s gốc.
- **Giá trị với đề tài:** paper nền rất sát với hướng **YOLO + lightweight + Jetson Nano**.

Paper 5 – The Development of a Waste Management and Classification System Based on Deep Learning and Internet of Things

- **Chen et al., 2024.**

- Dùng edge computing + IoT cho hệ thống phân loại rác.
- Mô hình cải tiến từ **YOLOv7-tiny**, hướng đến môi trường thực tế có nhiều loại rác trong cùng ảnh.
- Báo cáo trên Jetson Nano khoảng **110 ms/inference và 9 FPS**.
- **Giá trị với đề tài:** cho thấy cần quan tâm đồng thời accuracy, số tham số, FLOPs và tốc độ thực tế trên Jetson.

Paper 6 – Deep Learning Based Smart Bin for Efficient Sorting of Recyclable, Non-Recyclable, and Compostable Materials

- **ICCIT 2024.**
- Prototype smart bin tự động chia rác thành recyclable, non-recyclable và compostable.
- So sánh YOLOv8s và YOLOv8l; paper báo cáo mAP@50 lần lượt khoảng 92,7% và 93,4%.
- YOLOv8s được chọn cho real-time vì nhẹ hơn.
- **Giá trị với đề tài:** củng cố lựa chọn một phiên bản YOLO nhỏ thay vì model lớn trên edge device.

Paper/Study 7 – A Survey of Smart Dustbin Systems Using the IoT and Deep Learning

- **Artificial Intelligence Review, 2024.**
- Survey tổng hợp các hướng smart bin kết hợp IoT và Deep Learning.
- **Giá trị với đề tài:** dùng cho phần tổng quan nghiên cứu, phân loại các hướng smart bin và xác định khoảng trống mà prototype của nhóm giải quyết.

---

3. Hướng kỹ thuật đề xuất

3.1. Classification hay Object Detection?

Có hai phương án.

Phương án A – Image Classification

```
Waste image
    ↓
MobileNet / EfficientNet
    ↓
Plastic / Paper / Metal / Other
```

Ưu điểm: đơn giản, nhẹ và dễ chạy trên Jetson Nano.

Nhược điểm: giả định mỗi lần chỉ có một vật rác và vật nằm rõ trong vùng chụp.

Phương án B – Object Detection – đề xuất chọn

```
Image
   ↓
YOLO
   ↓
Bounding box + class + confidence
   ↓
Waste category
```

Phù hợp hơn cho đồ án vì:

- Có thể xác định rác nằm ở đâu.
- Chịu được vị trí vật thay đổi tốt hơn classification.
- Có thể mở rộng nhiều vật trong một ảnh.
- Demo trực quan bằng bounding box.
- Có cơ sở để đánh giá bằng Precision, Recall, mAP.
- Có nhiều paper liên quan trực tiếp đến YOLO + Jetson Nano.

3.2. Mô hình

Baseline đề xuất:

YOLOv5n / YOLOv8n → chọn model tốt nhất → ONNX → TensorRT FP16 → Jetson Nano

Không nên bắt đầu bằng model quá lớn. Jetson Nano là edge device hạn chế tài nguyên nên mục tiêu của đề tài không chỉ là accuracy cao mà còn phải đạt inference đủ nhanh.

---

4. Pipeline hoàn chỉnh

4.1. Pipeline phát triển AI

```
TACO / public datasets
          +
Dữ liệu tự thu bằng camera
          ↓
      Data Cleaning
          ↓
   Gộp nhãn thành 4 lớp
          ↓
       Annotation
          ↓
 Train / Validation / Test
          ↓
    Data Augmentation
          ↓
 Train YOLOv5n / YOLOv8n
          ↓
       Evaluation
          ↓
 Chọn mô hình phù hợp nhất
          ↓
       Export ONNX
          ↓
   TensorRT FP16
          ↓
      Jetson Nano
```

4.2. Pipeline hoạt động của thùng

```
Người dùng đưa rác vào
            ↓
IR / Ultrasonic Sensor
phát hiện có vật thể
            ↓
Camera chụp frame
            ↓
       Jetson Nano
            ↓
      Pre-processing
            ↓
       YOLO inference
            ↓
Class + Confidence + Bounding Box
            ↓
Kiểm tra confidence threshold
       ↙               ↘
Không chắc chắn        Hợp lệ
```

```
    ↓                    ↓
Other / Reject      Waste Category
                         ↓
                Gửi lệnh qua Serial
                         ↓
                    MCU/Arduino
                         ↓
                    Servo Motor
                         ↓
               Cửả/khay phân luồng
                  ↙   ↓   ↓   ↘
               Plastic Paper Metal Other
                         ↓
                Cảm biến mức đầy
                         ↓
                    Logging/UI
```

4.3. Logic một chu kỳ phân loại

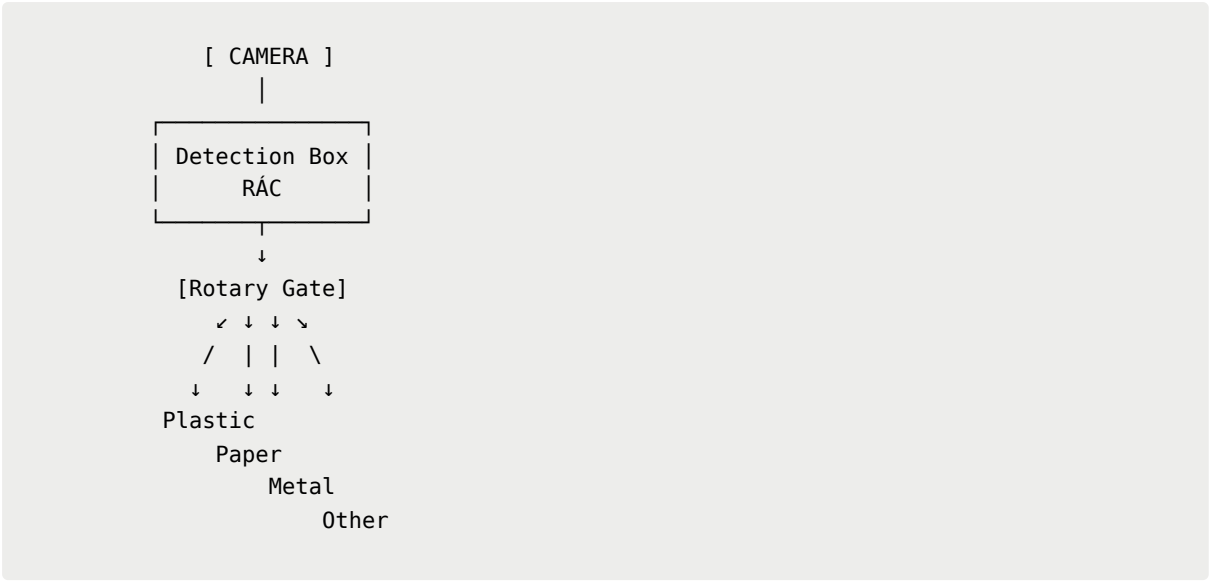
```
IDLE
 ↓
Object detected
 ↓
CAPTURE
 ↓
INFERENCE
 ↓
confidence >= threshold?
 ├── No  → OTHER / ERROR
 └── Yes
       ↓
   SELECT BIN
       ↓
   MOVE SERVO
       ↓
   DROP WASTE
       ↓
   SERVO HOME
       ↓
      IDLE
```

Điểm quan trọng là hệ thống **không cần camera inference liên tục**. Cảm biến có thể kích hoạt camera/inference khi có vật, giúp giảm tải và tránh phân loại lặp.

---

5. Thiết kế cơ khí đề xuất

Không cần robotic arm ở phiên bản đầu. Cơ cấu đơn giản và ổn định hơn là **khay/cửa xoay phân luồng**.



Jetson không trực tiếp cấp tải cho servo. Jetson xử lý AI và gửi class ID cho Arduino/ESP32; vi điều khiển đảm nhiệm servo và cảm biến.

---

6. Công nghệ sử dụng

Deep Learning / Computer Vision

- Python
- PyTorch
- YOLOv5/YOLOv8
- OpenCV
- Albumentations hoặc augmentation tích hợp của YOLO
- Roboflow/CVAT/LabelImg cho annotation

Edge AI

- NVIDIA Jetson Nano 4GB
- JetPack
- CUDA
- cuDNN
- TensorRT

- ONNX

Embedded

- Arduino Uno/Nano hoặc ESP32
- UART/Serial giữa Jetson và MCU
- Servo motor
- IR hoặc HC-SR04
- Cảm biến đo mức đầy nếu có

Quản lý mã nguồn/dữ liệu

- Git/GitHub
- Google Drive cho dataset lớn
- Jupyter/Google Colab hoặc GPU workstation để train

> **Lưu ý:** không nên train mô hình chính trên Jetson Nano. Train trên GPU/Colab, sau đó export và deploy inference trên Jetson.

---

7. Trang thiết bị cần chuẩn bị

7.1. Thiết bị bắt buộc

Thiết bị	SL	Vai trò
NVIDIA Jetson Nano 4GB	1	Edge AI inference
Camera USB hoặc CSI	1	Thu hình rác
Arduino/ESP32	1	Điều khiển cảm biến/servo
Servo torque phù hợp	1–2	Điều khiển cửa/khay phân loại
IR sensor hoặc HC-SR04	1	Phát hiện rác được đưa vào
Nguồn Jetson ổn định	1	Cấp nguồn Jetson
Nguồn riêng cho servo	1	Tránh servo làm sụt nguồn Jetson
Khung/thùng prototype	1	Hệ thống cơ khí
4 ngăn chứa	4	Chứa 4 nhóm rác
Dây, breadboard/PCB, connector	–	Kết nối

7.2. Nên có

- LED chiếu sáng cố định trong detection box.
- Màn hình nhỏ/LCD hoặc giao diện web hiển thị kết quả.
- Limit switch/cảm biến vị trí cho cơ cấu.
- Load cell nếu muốn phát triển thống kê khối lượng.
- Ultrasonic sensor cho từng ngăn nếu muốn đo mức đầy.

7.3. Thiết kế nguồn

```
AC Adapter / Battery
        │
        ├── 5V ổn định ──→ Jetson Nano
        │
        ├── 5V/3.3V ─────→ MCU + Sensors
        │
        └── Servo Supply ─→ Servo
```

Không cấp servo trực tiếp từ Jetson Nano.

---

8. Dataset

8.1. Nguồn public

TACO là nguồn ưu tiên cho object detection. Dataset chứa ảnh rác trong nhiều bối cảnh thực và annotation theo COCO.

Ngoài ra có thể khảo sát TrashNet hoặc các waste datasets trên Kaggle/Roboflow, nhưng phải kiểm tra license và mapping class trước khi đưa vào bộ dữ liệu chính thức.

8.2. Dữ liệu tự thu

Đây là phần quan trọng.

Nhóm nên đặt camera vào đúng **detection box** của prototype rồi tự chụp:

- nhiều loại chai nhựa;
- lon;
- giấy/carton;

- các vật Other;
- nhiều góc xoay;
- vật mới/cũ;
- vật bị bóp méo;
- nhiều điều kiện ánh sáng;
- tay người xuất hiện trong frame.

Lý do: paper về intelligent recycling bins đã chỉ ra dataset nền sạch như TrashNet có thể không đại diện tốt cho môi trường triển khai thực tế.

8.3. Chia dữ liệu

Baseline:

```
Train      70%
Validation 20%
Test       10%
```

Test set phải được giữ riêng và không dùng để tuning.

---

9. Thực nghiệm cần thực hiện

Experiment 1 – Baseline models

So sánh ít nhất:

- YOLOv5n
- YOLOv8n

Metrics:

- Precision
- Recall
- F1-score
- mAP@50
- mAP@50:95

Experiment 2 – Public data vs domain-specific data

```
Model A
Public dataset only

vs.

Model B
Public dataset + ảnh tự chụp
```

Mục tiêu: kiểm tra việc bổ sung ảnh đúng môi trường prototype có cải thiện khả năng nhận diện thực tế hay không.

Experiment 3 – Jetson optimization

```
PyTorch/FP32
     ↓
ONNX
     ↓
TensorRT FP16
```

Đo:

- inference latency;
- FPS;
- RAM;
- GPU utilization;
- model size.

Experiment 4 – End-to-end system

Không chỉ đánh giá AI.

Ví dụ 100–200 lần bỏ rác thực tế:

```
Waste
 ↓
AI correct?
 ↓
Servo correct?
 ↓
Correct bin?
```

Metrics:

Sorting Success Rate

```
Correctly sorted items
---------------------- × 100%
Total test items
```

và:

End-to-End Latency

```
t = t_detection + t_inference + t_control + t_mechanical
```

---

10. Tiêu chí hoàn thành

AI

- Nhận diện được 4 nhóm rác.
- Có confusion matrix.
- Báo cáo Precision/Recall/F1/mAP.
- Có benchmark model trước/sau optimization.

Edge AI

- Inference trực tiếp trên Jetson Nano.
- Không phụ thuộc cloud trong quá trình phân loại.
- Đo FPS và latency thực tế.

Hardware

- Cảm biến phát hiện rác.
- Camera thu ảnh.
- Servo tự chọn đúng ngăn.
- Cơ cấu trở về vị trí ban đầu sau mỗi lần phân loại.

System

- Demo được một chu kỳ hoàn chỉnh:

```
Đưa chai nhựa
     ↓
Plastic – 0.94
     ↓
Servo quay
     ↓
Chai rơi vào Plastic Bin
     ↓
Servo reset
```

---

11. Phân chia công việc cho 4 thành viên

> Mỗi người có **module chính**, nhưng không nên tách thành 4 project độc lập. Tất cả phải cùng tham gia integration và testing.

Thành viên	Vai trò chính	Công việc
TV1 – AI/Data	Dataset & Training	Tổng hợp dataset; mapping class; thu ảnh; annotation; augmentation; train YOLO; confusion matrix; Precision/Recall/mAP
TV2 – Edge AI	Jetson Deployment	Cài JetPack; camera pipeline; export ONNX; TensorRT FP16; inference app; benchmark FPS/latency/RAM
TV3 – Embedded/Hardware	Electronics & Control	Arduino/ESP32; sensor; servo; nguồn; UART; firmware; thiết kế logic actuator
TV4 – Mechanical/System	Prototype & Integration	Thiết kế thùng; detection box; rotary gate; lắp ráp; integration; test end-to-end; thu số liệu sorting

Công việc chung

- Đọc paper và viết related work.
- Xác định 4 class cuối cùng.
- Thiết kế test cases.
- Thực hiện end-to-end experiments.
- Viết báo cáo.
- Làm slide/video/demo.

Interface giữa các thành viên

Cần thống nhất ngay từ đầu:

```
TV1
trained_model.pt
       ↓
TV2
inference → class_id
       ↓
UART
       ↓
TV3
class_id → servo_angle
       ↓
TV4
mechanism → correct bin
```

Ví dụ protocol:

```
0 = OTHER
1 = PLASTIC
2 = PAPER
3 = METAL
```

Như vậy mỗi module có thể phát triển độc lập trước khi ghép hệ thống.

---

12. Timeline đề xuất

Giai đoạn 1 – Nghiên cứu & thiết kế

- Đọc các paper chính.

- Chốt 4 lớp rác.
- Chốt detection hay classification.
- Thiết kế sơ đồ phần cứng.
- Thiết kế cơ cấu phân luồng.

Output: proposal + architecture.

Giai đoạn 2 – Dataset

- Download public dataset.
- Mapping labels.
- Thu ảnh prototype.
- Annotation.
- Train/val/test split.

Output: dataset v1.

Giai đoạn 3 – AI

- Train YOLOv5n.
- Train YOLOv8n.
- Evaluate.
- Error analysis.
- Bổ sung dữ liệu.

Output: model v1.

Giai đoạn 4 – Hardware

- Camera.
- Sensor.
- Servo.
- MCU.
- Rotary gate.
- Prototype thùng.

Output: hardware v1.

Giai đoạn 5 – Edge Deployment

- Deploy model lên Jetson Nano.
- Camera real-time.
- TensorRT.
- Serial communication.

Output: inference v1.

Giai đoạn 6 – Integration

```
Sensor → Camera → AI → MCU → Servo → Bin
```

Output: prototype hoàn chỉnh.

Giai đoạn 7 – Evaluation

- AI metrics.
- Jetson benchmark.
- Sorting success.
- End-to-end latency.
- Failure cases.

Giai đoạn 8 – Báo cáo & demo

- Report.
- Slide.
- Poster/video nếu cần.
- Demo script.

---

13. Điểm nghiên cứu nên tập trung

Không nên đặt novelty là:

> "Dùng YOLO để phân loại rác."

Hướng đó đã có nhiều công trình.

Đề tài nên tập trung vào câu hỏi:

> **Một mô hình object detection nhẹ có thể đạt sự cân bằng như thế nào giữa độ chính xác và tốc độ khi triển khai hệ thống phân loại rác tự động hoàn toàn trên Jetson Nano?**

Có thể mở rộng thành ba câu hỏi:

RQ1. YOLOv5n và YOLOv8n khác nhau như thế nào về accuracy và inference speed trên Jetson Nano?

RQ2. Bổ sung dữ liệu tự thu trong môi trường thùng rác thực có cải thiện khả năng generalization so với chỉ dùng public dataset không?

RQ3. TensorRT FP16 cải thiện latency/FPS bao nhiêu và ảnh hưởng thế nào đến chất lượng dự đoán?

Như vậy phần AI của đề tài có **baseline** → **experiment** → **comparison** → **deployment** → **conclusion**, thay vì chỉ lắp một thùng rác chạy YOLO.

---

14. Rủi ro và phương án xử lý

Jetson Nano chạy chậm

→ Chọn YOLO nano/small, giảm input resolution, TensorRT FP16.

Dataset public không giống camera thật

→ Thu domain-specific dataset bằng chính prototype.

Servo hoạt động sai khi AI prediction dao động

→ Chỉ quyết định sau nhiều frame hoặc yêu cầu confidence vượt threshold.

Nhiều vật cùng lúc

→ Phiên bản đầu yêu cầu **mỗi lần một vật**. Multi-object sorting là hướng mở rộng.

Rác che khuất hoặc biến dạng

→ Data augmentation + bổ sung ảnh thực tế.

Cơ cấu bị kẹt

→ Detection box giới hạn kích thước vật + limit switch + timeout.

---

15. Hướng mở rộng sau khi baseline hoạt động

- Phân loại **organic/inorganic/recyclable**.
- Nhiều vật trong cùng frame.
- Instance segmentation.
- Load cell đo khối lượng.
- Cảm biến mức đầy.
- Dashboard thống kê.
- IoT gửi cảnh báo thùng đầy.
- Nhận diện vật không chắc chắn và chuyển sang ngăn reject.
- Continual data collection: lưu các ảnh confidence thấp để bổ sung dataset.
- So sánh Jetson Nano với thiết bị edge khác.

Tài liệu tham khảo khởi đầu

1. White, G., Cabrera, C., Palade, A., Li, F., & Clarke, S. **WasteNet: Waste Classification at the Edge for Smart Bins.** 2020.
2. Proença, P. F., & Simões, P. **TACO: Trash Annotations in Context for Litter Detection.** 2020.
3. Li, X., & Grammenos, R. **Evaluation of Practical Edge Computing CNN-based Solutions for Intelligent Recycling Bins.** IET Smart Cities, 2023.
4. Chen, Y. J., et al. **Classification and Recycling of Recyclable Garbage Based on Deep Learning.** Journal of Cleaner Production,
    a.
5. Chen, Z., et al. **The Development of a Waste Management and Classification System Based on Deep Learning and Internet of Things.** Environmental Monitoring and Assessment, 2024.
6. **Deep Learning Based Smart Bin for Efficient Sorting of Recyclable, Non-Recyclable, and Compostable Materials.** ICCIT 2024.

7. Arthur, M. P., Shoba, S., & Pandey, A. **A Survey of Smart Dustbin Systems Using the IoT and Deep Learning.** Artificial Intelligence Review, 2024.

Checklist bắt đầu ngay

- [ ] Chốt 4 class rác.
- [ ] Kiểm tra Jetson Nano và camera hiện có.
- [ ] Download TACO và khảo sát class distribution.
- [ ] Tìm thêm dataset phù hợp.
- [ ] Thiết kế detection box.
- [ ] Chọn YOLOv5n + YOLOv8n làm baseline.
- [ ] Chuẩn bị 30–50 vật rác thật để thu dữ liệu.
- [ ] Chụp dataset thử nghiệm.
- [ ] Thiết kế rotary gate.
- [ ] Test servo bằng MCU.
- [ ] Train model baseline.
- [ ] Deploy inference thử lên Jetson.
- [ ] Sau đó mới chế tạo prototype hoàn chỉnh.